

리먼 기선순기예우산능 | 스에신약에비 능신약약 o백에본백산 기물예술 섀
기선순기예우산

DNzw각 Nzw각Nzw각wV각k간츠각 zs각2ds개s a간간개개Nz간h 건
5 DgXOM868: .a간www|각개간겨wgXOMP각간범가각7g간47A2Z간478/
D 7:
D.N/ 5

미각 wd 거말3g		미각 wd 거말v		4
	2dbX			4
	e간랑/	2_cOPc Mk 1 XY Tē		견랑x
4			2	Qs 가간z 각범t
		4		
D				
74e간랑/	D_cOPc Mk 1 XY Tē			
84	D	2		
94	DQs 가간SZdh			
	D미각 wd 거말3g	견랑x	4	P것간거말
	2	4P것간거말		미각 wd 거말3g
e간랑/	2	OM	4	

P7D b기선순기예우산 i

미각 wd 거말v	Y w가dbX/ _Xea.	/ _XLa.	/	4
		2		D
	D			
3	.Y s가간Xwsx/	D	2	
	.c간랑거말/ 1	.N간간가각거말/	D	2
	.가간간t wdwy가각거말			
	.dzs잘 각/D			
미각 wd 거말3g	미각 wd 거말v		2	4Y 결개
OM	2 dbX	dbX	4	

P7 Db논삽4

미각 wd 거말3g		e간랑/	4	2
	4			
3	4			
	견랑x	dbX	D	

```
SELECT product_id, name, distance
FROM products
ORDER BY embedding <-> query_vector
```

LIMIT 10;

74Q₁ → D₁ . D₁ /
84d₁ → D₁ . D₁ /
94X₁ → D₁ . D₁ /

모든 거리: [5.2, 0.3, 2.1, 0.1, 3.4, ... 백만 개]
↓ (정렬)
정렬 결과: [0.1, 0.3, 2.1, 3.4, 5.2, ... 백만 개]
↓ (상위 10개)
결과: [0.1, 0.3, 2.1, 3.4, 5.2, ...]

D₁ . D₁ /
D₁ . D₁ /
D₁ . D₁ /
D₁ . D₁ /

가장 가까운 이웃을 찾는 방법

각각의 데이터 포인트에 대해, 가장 가까운 이웃을 찾는 방법
D₁ . D₁ /

Top() 연산자:
- 벡터 인덱스 탐색 (HNSW, IVF 등) → 후보 벡터를 k개 정도만 찾음
- 가져온 벡터들의 거리만 계산 → 그 중 상위 k개를 반환

74Q₁ → D₁ . D₁ /
84d₁ → D₁ . D₁ /
94X₁ → D₁ . D₁ /

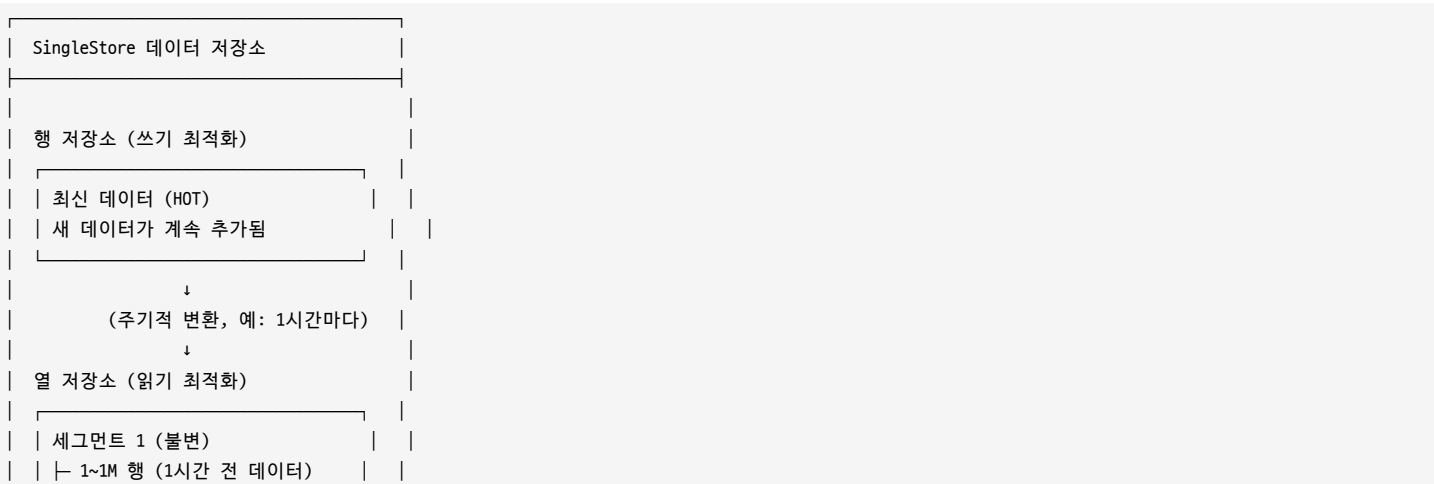
인덱스 탐색 경로:
쿼리 벡터에서 시작
↓
가까운 노드들 방문 (HNSW 그래프 탐색)
↓
거리 계산 (선택된 후보들만)
↓
상위 k개 추출

D₁ . D₁ /
D₁ . D₁ /
e₁ → D₁ . D₁ /

P₁ → I₁ b

각각의 데이터 포인트에 대해, 가장 가까운 이웃을 찾는 방법

가장 가까운 이웃을 찾는 방법



관악기산책술산쓰예

n악연기산술산쓰예

Top 10 검색:

↓
(결과 병합)
↓

D

P₂S Pb

SingleStore-V 인덱스 선택:

└ IVF_PQFS (빠른 검색)	
2. hnswlib 기반 인덱스	
└ HNSW_FLAT (전체 벡터 저장)	
└ HNSW_PQ (양자화 벡터)	
3. DiskANN (선택)	
└ SSD 기반 대규모 벡터 검색	

D

계	계	계	계	계	계
ᄒQqQXLe					1
ᄒQqab					1
SZdh					1
이개LZZ					

NcPLeP ZOPi

D

```
-- IVF 기반 인덱스 (메모리 사용 중간, 정확도 높음)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='IVF_FLAT', num_partitions=1000);

-- HNSW 기반 인덱스 (메모리 사용 많음, 속도 빠름)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='HNSW', max_connections=16);

-- PQ 기반 인덱스 (메모리 사용 적음, 빠름)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='IVF_PQ', num_partitions=1000, code_size=8);
```

P기

Db 예시

```
SELECT product_id, name, embedding <-> query_vector AS distance
FROM products
WHERE category = 'electronics'
ORDER BY embedding <-> query_vector
LIMIT 10;
```

디스크 기반

D

```
Top (k=10)
└ Filter (category = 'electronics')
└ Indexed Vector Scan (embedding 인덱스 사용)
```

D

74 us기

84

94 기

: 4 기

I b 6

```
SELECT p.product_id, p.name, p.price, p.embedding <-> query_vector AS sim
FROM products p
WHERE p.price BETWEEN 10000 AND 100000
AND p.rating > 4.0
AND p.embedding <-> query_vector < 1.0
ORDER BY p.embedding <-> query_vector
LIMIT 10;
```

D

```
Top (k=10)
└─ Filter (price BETWEEN 10000 AND 100000)
└─ Filter (rating > 4.0)
└─ Filter (distance < 1.0)
└─ Indexed Vector Scan (embedding 인덱스)
```

P2

디랙트 접근 방식은 GPU 메모리 사용량을 줄여줍니다.

예를 들어, 쿼리 D는 다음과 같습니다.

D: `SELECT * FROM products WHERE price > 78000 AND rating > 4.0 AND distance < 1.0`

이 쿼리는 `bad` 인덱스를 사용하여 처리됩니다.

이 쿼리는 `Y | 컬럼 OM2 /`로 처리됩니다.

강제 접근 방식은 `강제 접근 방식 dbX 2 OM /`로 처리됩니다.

OM.h vs 강제 접근 방식 /

D

	bad		
계33333333계33333333계33333333계33333333계			
Y 컬럼	8266	CC+	74RM
디랙트 접근 방식	82 66	CB4+	74RM
강제 접근 방식	B66	C>+	84RM
h vs 강제 접근 방식	7866	CA+	74RM

D

디랙트 접근 방식 Y | 컬럼

강제 접근 방식 9

P1

디랙트 접근 방식은 `강제 접근 방식`보다 4배 더 빠릅니다.

% 2 4

P2 접근 방식 D

디랙트 접근 방식 D

```
-- 단일 테이블 top-k 검색에 최적화
SELECT * FROM products
WHERE embedding <-> query_vector < threshold
ORDER BY embedding <-> query_vector
LIMIT 10;
```

P2 접근 방식 D

```
-- 여러 테이블의 복잡한 조인
SELECT o.order_id, p.product_id, p.name
FROM orders o
JOIN products p ON o.product_id = p.product_id
WHERE o.order_date > '2024-01-01'
AND p.embedding <-> query_vector < 1.0
ORDER BY p.embedding <-> query_vector
LIMIT 10;
```

D

74디랙트 접근 방식 e간행/ 2

84P2 접근 방식 디랙트 접근 방식 e간행/

94 D P2 접근 방식 %강제 접근 방식 766 % 2 디랙트 접근 방식

74		Dd 각 wd 거말8g	2		
		4	2		4
84		D NcPLeP ZOPi		4	
	2			4	
94		D	2	.	/
		4			
: 4		D 2dd02S00	2		
	4				
<4		Dd 각 wd 거말8g	2		
		4P 갖 거말		4	